

Departamentul Automatică și Informatică Industrială



**Facultatea Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie**

POLITEHNICA din București

Splaiul Independenței 313, 060042, București, România

Sala ED 412, Tel. 021/402.92.69

www.aii.pub.ro, email: secretariat.aii@upb.ro



Sesiunea de Comunicări Științifice Studențești

Ediția 2026

Compozitor Muzical al Traficului de Rețea

Autor: Nedelcu Bianca-Nicoleta, Automatică și Calculatoare, anul III, grupa 332AA

Adresa e-mail: bianca.nedelcu@stud.acs.upb.ro

Îndrumător științific: Conf. Dr. Ing. Mocanu Ștefan-Alexandru

Cuprins:

1. Introducere.....	3
2. Descrierea problemei și prezentarea soluției propuse	3-4
3. Prezentarea aplicației	4-11
3.1. Arhitectura	4-6
3.2 Parsarea capturii.....	6-7
3.3 Sonificarea traficului	7-9
3.4 Detectarea anomaliilor.....	9-10
3.5 Vizualizarea grafică	10-11
4. Concluzii si perspective de extindere	11-12
5. Bibliografie	13

1. Introducere

În ultimii ani, analiza traficului a devenit o activitate vitală în administrarea rețelelor și în securitate, din cauza atacurilor cibernetice la scară înaltă. Însă, în toate aceste contexte, analiza pachetelor de rețea se desfășoară aproape exclusiv prin reprezentări vizuale. Instrumente precum Wireshark sau tcpdump afișează liste lungi de pachete, iar utilizarea unui singur canal de percepție, vederea, devine o limitare atunci când capturile sunt mari.

Lucrarea aceasta prezintă o aplicație web numită Compozitor Muzical al Traficului de Rețea, care propune o abordare diferită pentru analiza traficului de rețea. Utilizatorul încarcă o captură, iar aplicația o transformă într-o reprezentare audio-vizuală sincronizată. În paralel cu sonificarea pachetelor, aplicația include și un modul de detectare automată a două tipuri de atacuri cibernetice: Denial of Service și Port Scanning (scanări de porturi).

Ideea de a folosi sunetul pentru a comunica informație este numită în literatura de specialitate sonificare [3] și este folosită în mai multe domenii. Un exemplu relevant în acest sens este în medicină: electrocardiograful traduce activitatea inimii într-un sunet care se modifică odată cu ritmul cardiac, iar personalul medical recunoaște auditiv abateri de la ritmul constant înainte de a privi monitorul. Aplicațiile de sonificare nu înlocuiesc reprezentarea vizuală, ci o completează: urechea umană detectează rapid abateri de la un ritm constant, chiar și atunci când atenția este îndreptată în altă parte.

2. Descrierea problemei și prezentarea soluției propuse

Problema pe care această aplicație încearcă să o rezolve este următoarea: cum poate fi prezentat conținutul unei capturi de rețea astfel încât evenimentele importante să poată fi sesizate rapid, fără parcurgerea manuală a fiecărui pachet? Scopul aplicației este de a ușura munca specialiștilor de securitate. În loc să urmărească vizual mii de pachete pentru a identifica zonele problematice, ei pot asculta captura și pot recunoaște auditiv momentele în

care apare trafic neobișnuit, fiind îndrumați direct către intervalele de timp care necesită atenție.

Soluția propusă pornește de la principiul că aceeași captură de trafic poate fi redată simultan, atât sonor, cât și vizual, concept abordat în literatura de specialitate pentru monitorizarea proceselor complexe [4]. Pentru ca această dublă reprezentare să fie utilă, ea respectă câteva reguli: fiecare protocol primește un instrument sonor distinct, traficul normal este redat printr-o muzică armonioasă, iar evenimentele anormale produc un sunet care contrastează puternic cu fondul muzical, reprezentând un zgomot. În același timp, vizualizarea grafică reflectă cu certitudine ce se aude, iar zonele de atac sunt marcate cu o culoare distinctă pentru ca specialistul să poată verifica imediat sursa alertei și momentul de timp.

Această abordare se aliniază cu funcțiile descrise de NIST Cybersecurity Framework, un cadru standardizat care, în versiunea 2.0, organizează activitățile de securitate cibernetică în șase etape: Govern, Identify, Protect, Detect, Respond și Recover [1]. Aplicația corespunde funcției Detect (identificarea evenimentelor suspicioase din monitorizarea traficului de rețea), iar marcarea sonoră și vizuală a zonelor anormale reprezintă punctul de tranziție către funcția Respond, în care specialistul poate continua analiza și aplica măsurile necesare de securitate.

Cele două tipuri de trafic malițios au fost alese din motive concrete. Acestea sunt printre cele mai frecvente forme de activitate ostilă într-o rețea, au semnături comportamentale clare care permit detectarea pe bază de reguli, fără modele de învățare automată [2] și produc sunete distincte atunci când sunt redată prin sonificare.

3. Prezentarea aplicației

3.1 Arhitectura

Aplicația este construită ca un sistem client-server, împărțit în două componente: un backend scris în Python, care se ocupă exclusiv de parsarea fișierelor cu capturile de trafic și un frontend scris în JavaScript, care realizează întreaga experiență audio-vizuală în browserul utilizatorului. Cele două componente comunică prin endpoint-ul /api/captures al backendului, iar întreaga logică de sonificare, vizualizare și detecție a anomaliilor rulează pe partea clientului.

Această alegere arhitecturală menține serverul ușor și simplifică implementarea. Serverul are o singură responsabilitate: să primească fișierul și să returneze pachetele parsate. După aceasta, întreaga experiență audio-vizuală se desfășoară în browser, fără a mai fi nevoie de comunicare cu serverul. Astfel, redarea capturii nu este afectată de eventuale probleme de rețea sau de latență către server.

Backend-ul expune două endpoint-uri: unul de tip GET pentru verificarea stării serverului (/api/health) și unul de tip POST pentru încărcarea capturii (/api/captures). Validarea datelor din fișiere se face cu Pydantic, care definește modelele Packet, CaptureMetadata, ProtocolCounts și CaptureResponse, iar parsarea pachetelor este realizată cu Scapy [5], cea mai folosită bibliotecă Python pentru parsarea pachetelor de rețea.

Componentă	Tehnologie		Rol
Limbaj de programare	Python 3.12		Limbajul de implementare
Framework Web	FastAPI		Gestionează endpoint-urile REST
Validare Date	Pydantic		Definește modelele de date
Parsarea capturii de trafic	Scapy	Citește fișierele .pcap / .pcapng și extrage informațiile necesare din fiecare pachet	
Middleware	CORSMiddleware		Permite request-uri din frontend (localhost:5173)

Tabel 1. Tehnologiile folosite pentru backend

Codul backendului este organizat în trei fișiere: main.py (entry point FastAPI [6] și definirea rutelor), models.py (modele Pydantic) și parser.py (logica de parsare a capturii). Serverul rulează pe portul 8000.

Frontend-ul este construit ca o aplicație React [7], scrisă în JavaScript cu sintaxă JSX (JavaScript XML). Pentru rularea aplicației este folosit Vite, un instrument care compilează codul sursă și îl optimizează pentru execuția în browser. Pentru sinteza audio se utilizează Tone.js [8], iar pentru vizualizarea grafică se folosește D3.js [9]. Aceste două framework-uri au fost alese deoarece sunt cele mai mature opțiuni pentru web: Tone.js oferă sintetizatoare configurabile și un sistem precis de programare temporală bazat pe Web Audio API, iar D3 permite crearea de vizualizări SVG complexe cu control complet asupra fiecărui element grafic.

Componentă	Tehnologie	Rol
Limbaaj de programare	JavaScript (JSX)	Scrierea aplicației
Build	Vite 5.4.10	Rularea (port 5173) și compilarea
Framework UI	React 18.3.1	Construirea interfeței
Vizualizare	D3.js 7.9.0	Desenarea diagramei
Audio	Tone.js 15.1.22	Web Audio API, sintetizatoare per protocol
Stilizare	CSS & Google Fonts	Stilizarea aplicației

Tabel 2. Tehnologiile folosite pentru frontend

Structura fișierelor frontendului reflectă o separare clară între componente UI și logica de business. Componentele React sunt grupate în `src/components/` (Header, Footer, Upload, Player, Visualization), iar logica de sonificare, vizualizare și detecție este izolată în `src/features`.

Pentru dezvoltare, frontend-ul folosește un proxy intern către backend, configurat în `vite.config.js`, care permite ca cererile către `/api` să fie redirecționate către serverul Python pe portul 8000, fără configurări suplimentare de CORS (Cross-Origin Resource Sharing) în mediul de dezvoltare.

Fluxul de date prin aplicație este unidirecțional și se desfășoară în următoarea ordine. Utilizatorul încarcă un fișier PCAP sau PCAPNG prin interfața de upload, fie prin drag and drop, fie prin selectare manuală. Fișierul este validat local (extensie, conținut nevid) și apoi trimis către endpoint-ul `/api/captures`. Pe server, parsarea cu Scapy este executată într-un thread separat, astfel încât interfața să rămână responsivă chiar și pentru fișiere de dimensiuni mari, iar rezultatul este returnat ca JSON. Frontend-ul primește răspunsul, îl stochează în starea aplicației și inițializează cele trei etape paralele: programarea notelor muzicale pe baza pachetelor (cu Tone.js), desenarea vizualizării (cu D3.js) și aplicarea algoritmului de detecție a anomaliilor. Din acest punct, utilizatorul poate controla redarea prin butoanele Reda, Pauză și Stop, iar cele două canale (audio și vizual) rămân sincronizate prin folosirea aceluiași ceas, oferit de Tone.Transport.

3.2 Parsarea capturii

Atunci când backend-ul primește un fișier prin endpoint-ul `/api/captures`, primul pas este validarea extensiei, fișierul neputând fi gol. Dacă oricare dintre aceste condiții nu este îndeplinită, serverul răspunde cu o eroare HTTP 400. Dacă validarea trece, conținutul

fișierului este transmis bibliotecii Scapy într-un thread separat, astfel încât interfața să rămână responsabilă pe durata parsării, chiar și pentru fișiere PCAP de dimensiuni mari.

Pentru fiecare pachet din captură, aplicația extrage patru informații: timpul relativ față de primul pachet (calculat ca diferență dintre timestamp-ul curent și timestamp-ul primului pachet din captură), protocolul, porturile sursă și destinație (atunci când sunt aplicabile) și dimensiunea în octeți. Adresele IP, flag-urile TCP, payload-ul și alte câmpuri pe care Scapy le-ar putea oferi nu sunt extrase, fiindcă nu sunt necesare pentru sonificare sau pentru detecția anomaliilor.

Identificarea protocolului se face printr-o cascadă de verificări, în ordinea: ARP, ICMP, DNS, TCP, UDP. Această ordine permite ca un pachet să fie etichetat corect chiar și atunci când apare în mai multe straturi. De exemplu, un pachet DNS este transportat în mod obișnuit prin UDP, iar dacă verificarea s-ar face în ordine inversă (UDP înainte de DNS), toate pachetele DNS ar fi clasificate eronat drept UDP. Pentru a evita acest comportament, aplicația verifică explicit dacă pachetul este DNS, fie prin prezența stratului DNS în Scapy, fie prin existența portului 53 ca sursă sau destinație. Pachetele care nu se încadrează în niciuna dintre cele cinci categorii primesc eticheta OTHER.

În paralel cu extragerea informațiilor per pachet, aplicația calculează metadate agregate pentru întreaga captură: durata totală, numărul de pachete, dimensiunea minimă și maximă a pachetelor, distribuția pe protocoale și rata maximă de pachete pe secundă. Această ultimă valoare este obținută prin gruparea pachetelor în bucket-uri de câte o secundă și selectarea valorii maxime. Toate aceste metadate, împreună cu lista pachetelor, sunt împachetate într-un obiect JSON validat de modelele Pydantic și trimise către frontend. Pentru cazul în care captura nu are niciun pachet, serverul returnează un răspuns valid cu zero pachete și metadate puse pe zero, fără a afișa o eroare.

3.3 Sonificarea traficului

Sonificarea este componenta centrală a aplicației și transformă lista de pachete primită de la backend într-o secvență audio. Fiecare pachet este programat să producă un sunet la momentul corespunzător timpului său relativ, iar coordonarea între pachete este asigurată de obiectul Tone.Transport, care funcționează ca un ceas central pentru întreaga redare.

Asocierea fiecărui protocol cu un instrument, o culoare și o durată a fost o alegere subiectivă și intuitivă, ghidată de caracterul fiecărui protocol în rețea. Această abordare se încadrează în paradigma parameter mapping sonification, în care caracteristicile datelor sunt asociate cu parametri sonori precum timbru, durată și înălțime [3]. TCP, ca protocol fiabil și cel mai prezent în internet, a primit un sintetizator cu undă triunghiulară (sunet cald și clar) și culoarea violet (sugestie de stabilitate). UDP, rapid și fără confirmare, a primit un sintetizator de membrană (sunet scurt și rapid) și culoarea portocalie (energică), cu cea mai scurtă durată dintre toate protocoalele. ICMP, folosit pentru semnale tehnice precum ping-uri, a primit un sintetizator metalic (care evocă tocmai sunetul unui ping), fiind asociat cu albastru rece. DNS, omniprezent în trafic, a primit o undă sinusoidală pură (potrivită pentru interogări simple cerere-răspuns) și verde, asociată intuitiv cu noțiunea de validare. ARP, protocol rar, dar semnificativ la Layer 2, a primit un sintetizator de membrană cu durată lungă (fiecare apariție contează) și maro (nivel scăzut, aproape de hardware). Categoria OTHER a rămas pe undă sinusoidală, asociată culorii gri, ce semnifică aspect neutru.

Protocol	Instrument Tone.js	Culoare	Durăță Notă
TCP	Synth (Undă Triunghiulară)	Violet	8n
UDP	MembraneSynth	Portocaliu	16n
ICMP	MetalSynth	Albastru Rece	4n
DNS	Synth (Undă Sinusoidală)	Verde	4n
ARP	MembraneSynth	Maro	2n
OTHER	Synth (Undă Sinusoidală)	Gri	8n

Tabel 3. Maparea protocoalelor

Notățiile pentru durată sunt convențiile muzicale din Tone.js: 8n corespunde unei optimi, 16n unei șaisprezecimi, 4n unei pătrimi, iar 2n unei doimi. Cu cât numărul duratei este mai mare, cu atât sunetul este mai scurt. Aceste durate au fost alese astfel încât să reflecte natura protocoalelor: pachetele scurte și frecvente (UDP) primesc note scurte, în timp ce pachetele rare, dar semnificative (ARP) primesc note lungi, care le fac mai vizibile auditiv.

Înălțimea fiecărei note este determinată de portul destinație al pachetului, mapat pe scala pentatonică în Do major (C4, D4, E4, G4, A4, C5, D5, E5, G5, A5) prin operația modulo. Această scală a fost aleasă deoarece orice combinație de note din ea sună armonios, ceea ce evită oboseala auditivă în cazul unei redări prelungite. Volumul fiecărei note este proporțional cu dimensiunea pachetului, calculat printr-o mapare liniară între dimensiunea minimă și

maximă din captură pe intervalul de volum 0.2 - 0.8. Pachetele mai mari produc astfel note mai puternice, iar pachetele foarte mici se aud mai încet, fără a deveni dominante.

Atunci când o captură conține mii de pachete pe secundă, redarea simultană a tuturor notelor ar produce un zgomot indescifrabil și ar depăși capacitatea sistemului audio. Pentru a evita acest comportament, aplicația limitează numărul maxim de sunete redade într-o singură secundă la 8, acestea fiind selectate uniform, calculate printr-un pas constant.

Pentru secunde care sunt clasificate ca atac de către modulul de detectare, comportamentul este complet diferit: secunda este înlocuită cu un singur sunet de zgomot alb distorsionat, generat prin combinarea unui NoiseSynth cu un efect de distorsiune și un filtru bandpass centrat pe 1800 Hz. Volumul acestui zgomot crește logaritmicește cu numărul de pachete din secundă, astfel încât atacurile masive sună mai intens decât cele moderate.

3.4 Detectarea anomaliilor

Modulul de detectare identifică două tipuri de comportament malițios în trafic: atacurile de tip Denial of Service (DoS) și scanările de porturi. Modulul se bazează pe reguli cu praguri stabilite, abordare cunoscută în literatura de specialitate ca signature-based intrusion detection [2]. Analiza se desfășoară pe câte o secundă. Pachetele sunt mai întâi grupate în bucket-uri corespunzătoare secunde în care au fost capturate, prin aplicarea funcției Math.floor asupra timpului relativ al fiecărui pachet. Apoi, pentru fiecare secundă, sunt rulate două clasificări independente: una bazată pe volumul de trafic (pentru DoS) și una bazată pe numărul porturilor destinație (pentru port scanning). Severitatea finală a fiecărei secunde este maximul dintre cele două clasificări, conform ierarhiei normal < suspicious < attack.

Un atac Denial of Service este caracterizat printr-un volum mare de pachete trimise într-un timp scurt către o singură destinație. Modulul folosește două criterii pentru a identifica o astfel de secundă: numărul total de pachete și gradul de concentrare a traficului pe un singur port destinație.

Parametru	Valoare	Semnificație
ATTACK_PACKETS_PER_SECOND	100	Peste 100 pachete într-o secundă declanșează clasificarea „attack”
SUSPICIOUS_PACKETS_PER_SECOND	30	Între 30 și 100 pachete pe secundă: clasificare „suspicious”
FLOOD_SAME_PORT_RATIO	0.8	Dacă cel puțin 80% din pachete au același port destinație, este flood
FLOOD_MIN_PACKET_COUNT	10	Pragul minim de pachete pentru a aplica regula de flood

Tabel 4. Praguri pentru detectare Denial of Service

O scanare de porturi nu este caracterizată prin volum, ci prin diversitate: un atacator trimite pachete către multe porturi diferite ale aceleiași victime, pentru a descoperi care servicii sunt deschise. Aceste pachete pot fi puține și răspândite în timp, ceea ce înseamnă că o analiză strict per secundă ar putea rata scanările mai lente. Pentru a rezolva această problemă, modulul folosește o fereastră de cinci secunde, în care acumulează toate pachetele și numără porturile destinație unice.

Parametru	Valoare	Semnificație
PORT_SCAN_WINDOW_SECONDS	5	Lungimea ferestrei de analiză, în secunde
PORT_SCAN_SUSPICIOUS_PORTS	15	> 15 porturi unice în fereastră: clasificare „suspicious”
PORT_SCAN_ATTACK_PORTS	50	> 50 porturi unice în fereastră: clasificare „attack”

Tabel 5. Praguri pentru detectare Port Scanning

Pragurile prezentate în tabelele 4 și 5 sunt valori care funcționează bine pentru capturi de trafic obișnuit (rețele de birou, trafic personal), dar nu sunt fixe. Acestea pot fi ajustate pentru a se potrivi cu specificul rețelei analizate, fără a modifica logica algoritmului. De exemplu, într-un mediu de tip data center, unde traficul normal poate atinge sute de pachete pe secundă, pragul ATTACK_PACKETS_PER_SECOND poate fi crescut pentru a evita alarmele false. Invers, într-o rețea cu trafic foarte redus (de exemplu, dispozitive IoT cu comunicații rare), pragurile pot fi coborâte pentru a detecta atacuri de volum mic, care altfel ar trece neobservate.

3.5 Vizualizare grafică

Vizualizarea grafică oferă o reprezentare a întregii capturi sub forma unei diagrame cronologice. Biblioteca D3.js permite construirea de grafice SVG pentru fiecare element. Diagrama este desenată o singură dată, atunci când captura este încărcată, iar pe parcursul redării se actualizează doar poziția cursorului temporal și opacitatea pachetelor deja redade.

Pe axa orizontală este reprezentat timpul, exprimat în secunde, iar pe axa verticală sunt distribuite cele șase categorii de protocoale (TCP, UDP, ICMP, DNS, ARP, OTHER) sub forma a șase benzi paralele. Fiecare pachet apare ca un cerculeț poziționat la coordonatele corespunzătoare: timpul său determină poziția pe axa orizontală, iar protocolul determină banda în care este desenat. Culoarea cerculețului reflectă protocolul (conform Tabelului 3), iar

raza sa este proporțională cu dimensiunea pachetului, calculată printr-o scală de tip rădăcină pătrată între 2 și 6 pixeli. Această alegere a fost făcută pentru ca diferențele de dimensiune să fie vizibile fără ca pachetele mari să domine vizual diagrama.

Zonele de atacuri sunt reprezentate sub forma unor dreptunghiuri roșii. Intensitatea culorii și înălțimea dreptunghiului depind de severitate: secunde clasificate ca “attack” sunt marcate cu un roșu mai intens și acoperă toată înălțimea diagramei, în timp ce secunde clasificate ca “suspicious” sunt marcate cu un roșu mai deschis și acoperă doar jumătatea centrală. Această diferențiere permite utilizatorului să distingă vizual evenimentele certe și cele care necesită investigare suplimentară.

În timpul redării, un cursor vertical (numit playhead) se deplasează de la stânga la dreapta, sincronizat cu Tone.Transport printr-un mecanism bazat pe requestAnimationFrame. Poziția cursorului indică momentul curent din captură, iar pachetele aflate la stânga sa, deja redade, apar cu opacitate ridicată, în timp ce pachetele aflate la dreapta sa, cele neredate, apar cu opacitate scăzută.

4. Concluzii și perspective de extindere

Detectarea automată a traficului malițios este astăzi un domeniu matur [2]. Instrumente bazate pe semnături precum Snort, Suricata sau Zeek identifică atacuri cunoscute prin reguli predefinite, iar soluții comerciale precum Darktrace sau Vectra AI folosesc învățare automată pentru a semnală abateri de la tiparul normal al unei rețele.

Scopul lucrării a fost de a demonstra cât de important este un sunet de alertă atunci când se produc atacuri la nivel de rețea. Un sistem de inteligență artificială poate identifica o anomalie, cu precizie ridicată, dar dacă alerta este doar o linie într-un log sau un punct pe un grafic, eficiența reală depinde de capacitatea operatorului de a o observa la timp [4]. O alertă vizuală poate trece neobservată minute întregi, în care un atac deja produce daune.

Sonificarea oferă un canal complementar pe care instrumentele actuale îl exploatează doar marginal. Sistemele de tip SIEM produc notificări sonore generice (bip-uri sau alerte audio standard), dar nu folosesc sonificarea propriu-zisă, în care caracteristicile traficului sunt mapate sistematic pe parametri sonori. Există cercetare academică în această direcție, însă

niciun produs comercial larg adoptat nu o implementează. Aplicația dezvoltată în această lucrare nu se propune ca alternativă la sistemele de inteligență artificială existente, ci ca strat suplimentar de comunicare a alertelor către utilizator, complementar canalului vizual.

Proiectul a îndeplinit obiectivele stabilite la începutul lucrării și a livrat un prototip funcțional pe care l-am putut testa pe capturi reale. Aplicația se adresează în primul rând studenților care învață despre rețele și securitate cibernetică și care au nevoie să dezvolte o intuiție pentru diverse tipuri de trafic înainte de a stăpâni instrumentele profesionale (o scanare de porturi sună altfel decât un transfer de fișiere). În al doilea rând, este utilă analiștilor și cercetătorilor care pot folosi sonificarea ca un canal suplimentar de informație, complementar instrumentelor clasice.

Pornind de la această poziționare, există trei direcții concrete în care aplicația poate fi extinsă pentru a-și atinge potențialul deplin.

Captura live de trafic: Extinderea către captura directă a traficului de pe interfața de rețea ar transforma aplicația într-un instrument de monitorizare continuă, în care alertele sonore ar fi produse pe măsură ce atacurile se desfășoară, în timp real. Această direcție presupune adăugarea unei componente de sniffing activ cu Scapy și a unei conexiuni persistente între backend și frontend, prin care pachetele să fie transmise în timp real prin WebSocket.

Extinderea setului de protocoale: Adăugarea protocoalelor precum HTTP, HTTPS, TLS, SSH, SMB, SMTP sau NTP ar permite o reprezentare mai bogată a traficului, fiecare protocol primind propriul instrument sonor. Modificarea presupune extinderea funcției de identificare din backend și actualizarea schemei de mapare din frontend.

Detecția unui set mai larg de atacuri: Pe lângă Denial of Service și scanările de porturi, aplicația ar putea identifica atacuri ARP spoofing, exfiltrare prin DNS tunneling, atacuri de forță brută sau trafic către domenii malițioase. Fiecare tipar ar primi propria semnătură sonoră distinctă, permițând utilizatorului să distingă auditiv nu doar prezența unui atac, ci și tipul acestuia.

5. Bibliografie

- [1] National Institute of Standards and Technology. (2024). *The NIST Cybersecurity Framework (CSF) 2.0* (NIST Cybersecurity White Paper CSWP 29). Gaithersburg, MD: NIST.
- [2] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1), (pp 1-22).
- [3] Grond, F., & Berger, J. (2011). Parameter Mapping Sonification. În T. Hermann, A. Hunt, & J. G. Neuhoff (Eds.), *The Sonification Handbook* (pp. 363-397). Berlin: Logos Verlag.
- [4] Vickers, P. (2011). Sonification for Process Monitoring. În T. Hermann, A. Hunt, & J. G. Neuhoff (Eds.), *The Sonification Handbook* (pp. 455-491). Berlin: Logos Verlag.
- [5] D3.js Documentation. <https://d3js.org/> (ultima accesare: 15 mai 2026)
- [6] FastAPI Documentation. <https://fastapi.tiangolo.com/> (ultima accesare: 15 mai 2026)
- [7] React Documentation. <https://react.dev/> (ultima accesare: 15 mai 2026)
- [8] Scapy Documentation. <https://scapy.readthedocs.io/> (ultima accesare: 15 mai 2026)
- [9] Tone.js Documentation. <https://tonejs.github.io/> (ultima accesare: 15 mai 2026)